

# AGENTS.md

本文件是给 Codex / AI Coding Agent 读取的项目级规则。  
本项目不再按 Phase 推进开发，而是按 Capability、Design Spec、Test Case、Acceptance Criteria 推进。  
任何代码生成、重构、测试、文档更新都必须遵守本文件。

---

## 1. 项目基本原则

本项目是玩具制造业 ERP。

当前仍处于开发阶段，可以破坏旧实现，不需要兼容错误的旧 API 或旧命名。

正确开发单位是：

功能能力 Capability  
设计规格 Design Spec  
测试用例 Test Case  
验收标准 Acceptance Criteria  
交付资料 Delivery Package

错误开发单位是：

Phase  
阶段闭环  
模拟闭环  
历史阶段编号

不要再以 `Phase 8`、`Phase 9`、`Phase 10`、`Phase 11`、`Phase 12` 等方式组织新功能。

---

## 2. 禁止 Runtime Phase 命名

Phase 只能作为历史记录存在，不允许进入运行时代码。

### 2.1 禁止新增

禁止新增以下命名：

phase8  
phase9  
phase10

```
phase11
phase12
Phase8
Phase9
Phase10
PHASE8
PHASE9
/erp/phase8
/erp/phase9
phase8.createShipment
phase8.createFinanceFact
phase8.createReservation
Phase8Usecase
jsonrpc_phase8.go
phase8_repo.go
Phase8FactsPage
```

---

## 2.2 禁止出现的位置

Phase 命名不得出现在：

```
server/internal/
web/src/
web/scripts/
config/
deploy/
deployments/
数据库表名
JSON-RPC method
API route
前端 route
菜单名称
React component 名称
Go package 名称
测试名称
客户可见文档
部署文档
```

---

## 2.3 允许出现的位置

Phase 只允许出现在历史归档文档中：

```
docs/archive/  
docs/history/  
docs/evidence/
```

并且只能用于说明历史阶段，不能作为当前开发入口。

### 3. 使用正式业务领域命名

必须使用真实业务领域命名。

示例：

```
phase8.createShipment      -> shipment.create  
phase8.shipShipment        -> shipment.ship  
phase8.cancelShippedShipment -> shipment.cancelShipped  
phase8.createReservation   -> inventory.reserve  
phase8.cancelReservation   -> inventory.cancelReservation  
phase8.createFinanceFact   -> finance.createFact  
phase8.createProductionFact -> production.createFact  
phase8.createOutsourcingFact -> outsourcing.createFact  
/erp/phase8/facts          -> /erp/shipments  
/erp/phase8/finance        -> /erp/finance/facts  
Phase8Usecase              -> ShipmentUsecase / StockReservationUsecase /  
FinanceFactUsecase
```

不要保留 phase alias。

当前项目仍在开发阶段，允许直接破坏兼容并删除 phase 旧入口。

### 4. 新任务组织方式

不要说：

```
完成 Phase 13  
继续下一阶段  
完善 Phase 8  
做产品化阶段
```

应该使用：

```
CAP-XXX：功能能力  
TEST-XXX：测试用例
```

PAGE-XXX : 页面  
API-XXX : API 合同  
MIG-XXX : 数据库迁移  
DEPLOY-XXX : 部署交付

示例：

CAP-SHIP-001 : 出货单  
TEST-SHIP-001 : 重复 ship 不重复扣库存  
TEST-SHIP-002 : 重复 cancel shipped 不重复回滚  
CAP-RES-001 : 库存预留  
TEST-RES-001 : 并发预留不能超过可用库存  
CAP-BOM-001 : BOM Version  
TEST-BOM-001 : 同一 SKU 只能有一个 ACTIVE BOM  
CAP-PO-001 : 采购订单  
TEST-PO-001 : 采购收货不能超过采购数量

## 5. Codex 执行任务时必须先确认

每次实现功能前，必须先明确：

1. 这属于哪个 Capability？
2. 对应哪个 Test Case？
3. 修改哪个事实源？
4. 是否需要 migration？
5. 是否需要 audit event？
6. 是否需要权限检查？
7. 是否会影响库存、出货、采购、财务等关键事实？
8. 是否会引入 phase 命名？
9. 是否会写 business\_records？
10. 是否会绕过 biz 主路径？

如果任务没有明确 capability/test case，应该先补充设计或测试，不要直接大范围实现。

## 6. 必读文档

修改产品、架构、测试、部署、导入相关内容前，应优先阅读相关文档：

PROJECT\_HARDENING\_ROADMAP.md  
PRODUCT\_CORE\_MENU\_SPEC.md  
AUTOMATED\_TESTING\_PLAN.md  
TEST\_DATA\_STRATEGY.md

```
CLIENT_CONFIGURATION_SEED_IMPORT_DEPLOYMENT.md
SERVER_INTERNAL_CORE_LAYERING.md
YOYOSUN_PRIVATE_DEPLOYMENT_PACKAGE.md
docs/architecture/NO_PHASE_RUNTIME_POLICY.md
```

如果文档不存在，不要自行发明新阶段；按任务要求创建或更新对应文档。

## 7. 后端分层规则

目标分层：

```
server/internal/server
server/internal/service/jsonrpc
server/internal/biz
server/internal/core
server/internal/data
```

职责：

```
service/jsonrpc:
  解析参数
  鉴权
  权限检查
  调用 biz
  错误映射
  返回 DTO

biz:
  usecase
  事务编排
  idempotency
  audit event
  调用 core 规则
  调用 data repository

core:
  纯产品规则
  状态机
  值对象
  计算器
  领域错误
  无 IO
  无 DB
  无 JSON-RPC

data:
```

```
Ent
SQL
repository 实现
transaction
lock
persistence
```

禁止：

不要把 JSON-RPC dispatch 放进 server/internal/data。  
不要让 data 层变成 service 层。  
不要让 core 变成第二套 biz。  
不要让 service 直接绕过 biz 调 data。

## 8. core 层限制

server/internal/core 只能放：

```
纯领域规则
状态机
值对象
业务不变量
纯计算器
领域错误
领域事件定义
policy 类型
```

禁止 core import：

```
server/internal/data
server/internal/service
ent
database/sql
net/http
os.Getenv
Kratos transport
配置 loader
JSON-RPC DTO
```

禁止在 core 中出现：

```
CreateShipment
ShipShipment
```

```
ApplyImport
BootstrapAdmin
LoadCustomerConfig
SaveInventoryTxn
DispatchJSONRPC
```

这些属于 `biz`、`service` 或 `data`，不属于 `core`。

## 9. 业务事实源规则

每个业务事实必须只有一个事实源。

禁止用 `business_records` 写正式 ERP 事实。

`business_records` 只允许作为：

```
legacy archive
read-only historical viewer
migration bridge
```

禁止 `business_records` 承担：

```
新销售订单事实
新采购事实
新库存事实
新出货事实
新财务事实
新生产事实
新外协事实
```

正式事实源应使用：

```
sales_orders / sales_order_items
purchase_orders / purchase_order_items
purchase_receipts
quality_inspections
inventory_txns
inventory_balances
stock_reservations
production_facts
outsourcing_facts
shipments / shipment_items
finance_facts
```

```
workflow_tasks
audit_events
```

## 10. Workflow 边界

Workflow 只负责任务流，不负责正式业务事实。

Workflow 可以处理：

```
任务
审批
指派
评论
待办
workflow task 状态
业务对象引用
```

Workflow 不允许直接写：

```
inventory_txns
inventory_balances
shipments
shipment_items
finance_facts
purchase_receipts
production_facts
outsourcing_facts
```

如果 workflow 完成后需要改变业务事实，必须调用对应领域 usecase。

## 11. 测试优先规则

高风险 ERP 行为必须先补测试，或测试和实现一起提交。

必须覆盖：

```
库存不能负数
库存预留并发不能超预留
出货 ship 幂等
取消已发货出货幂等
采购收货不能超收
BOM ACTIVE 状态规则
```



同一 SKU 只能有一个 ACTIVE BOM  
财务事实同一来源不能重复生成  
business\_records 正式写入被拒绝  
runtime phase 命名被禁止  
workflow 不直接写业务事实  
后端 RBAC 强制执行  
关键动作写 audit event

不要只实现 happy path。

不要只写页面不写后端权限。

不要只写文档不写测试。

---

## 12. 安全规则

禁止提交：

真实 token  
真实 password  
JWT secret  
private key  
webhook secret  
Telegram bot token  
npm registry token  
.env  
生产数据库 URL  
客户 raw credential  
数据库 dump  
生产备份

只允许提交：

.env.example  
\*.example.yaml  
\*.example.json  
placeholder values  
CI secrets 引用说明  
server environment variables 说明  
Docker secrets 说明

## 13. 客户数据规则

禁止提交真实客户 raw files 到产品仓库或部署文档：

```
raw Excel
raw PDF
raw image
database dump
真实客户地址
真实手机号
真实订单明细
未脱敏财务金额
```

允许提交：

```
脱敏样例
source manifest
mapping files
hash
汇总报告
known limitations
不含 secret 的 deployment evidence
```

---

## 14. 测试数据规则

测试数据必须：

```
确定性
合成或脱敏
可重复执行
可清理
持久化时带 test_run_id
```

禁止测试数据进入：

```
production
customer-trial
private-deploy
```

库存测试数据必须通过库存流水创建，不能只直接改库存余额。

---

## 15. 完成任务后的输出要求

每次任务完成后必须输出：

1. 修改了什么
2. 修改了哪些文件
3. 新增或修改了哪些测试
4. 运行了哪些命令
5. 测试结果
6. 哪些测试没跑，原因是什么
7. 是否破坏兼容
8. 是否涉及 migration
9. 是否涉及安全
10. 剩余风险

不要静默跳过测试。

---

## 16. Review 检查清单

每个 PR 必须检查：

- ☐ 是否新增 runtime phase 命名。
- ☐ 是否保留 phase API alias。
- ☐ 是否新增 phase 前端路由。
- ☐ 是否用正式业务领域命名。
- ☐ 是否用 business\_records 写正式事实。
- ☐ workflow 是否直接写业务事实。
- ☐ JSON-RPC 是否进入 data 层。
- ☐ core 是否 import data/service/Ent/SQL/HTTP/config。
- ☐ 关键动作是否有测试。
- ☐ 关键动作是否有 audit event。
- ☐ 权限是否由后端强制。
- ☐ 是否提交 secret。
- ☐ 是否提交客户 raw data。
- ☐ 测试是否实际运行。

---

## 17. 一句话原则

按功能能力和测试推进，不按 Phase 推进。

更具体：

Phase 只记录历史。

Capability 才指导开发。

Test Case 才证明完成。

Acceptance Criteria 才能验收。